

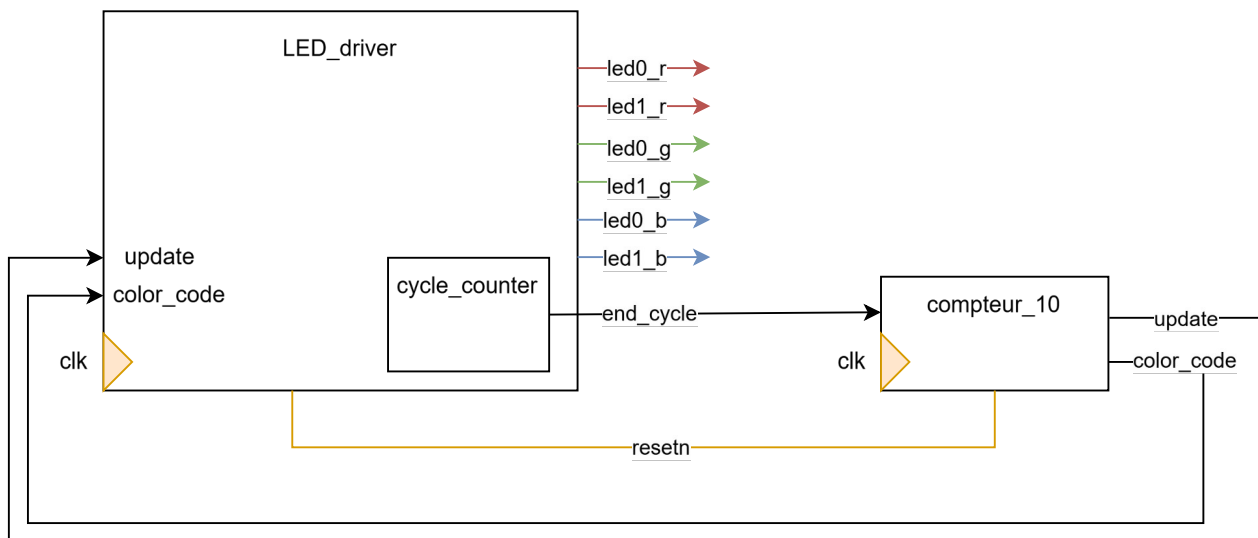
TP5 – Domaines d'horloge Rev

Objectif : L'objectif de ce TP est de mettre en place une architecture utilisant plusieurs domaines d'horloge. Pour cela, vous utiliserez deux LEDs RGB qui clignoteront avec des fréquences différentes grâce aux horloges. Vous apprendrez également à utiliser une PLL pour générer des horloges

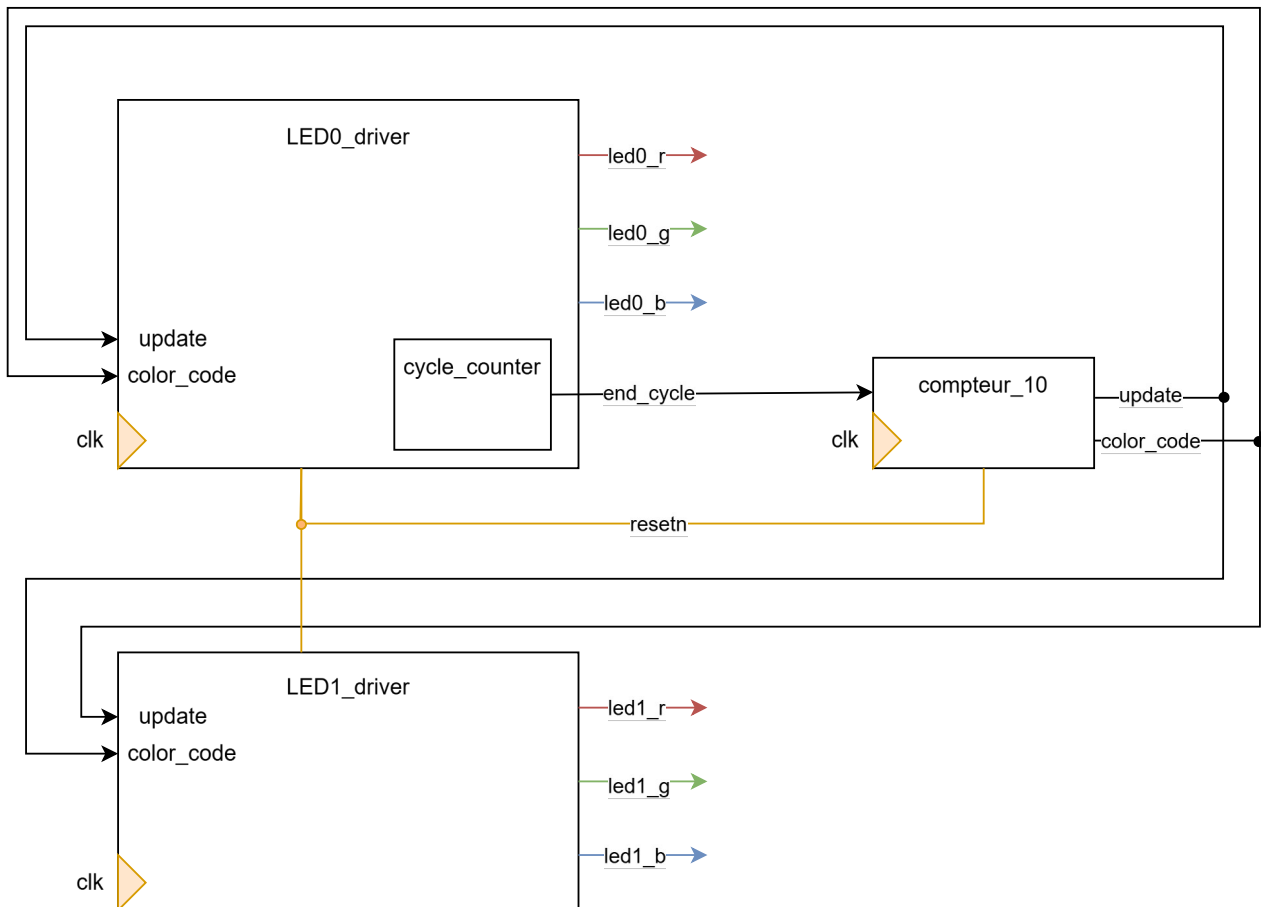
1. A l'aide du module LED_driver du TP4, créez une architecture RTL permettant de piloter les deux LED RGB. Les LEDs RGB devront clignoter 10 fois en rouge puis 10 fois en bleu et 10 fois en vert avant de recommencer à partir du rouge. En entrée des modules LED_driver le signal update ne devra pas être à 1 pendant plus d'un coup d'horloge. Il doit s'agir d'une impulsion.

Le compteur de temporisation devra compter 100 000 000 coups d'horloge (pour une fréquence d'horloge à 100MHz, cela correspond à 1s). Dans la suite de ce TP, la valeur du compteur de temporisation ne sera pas modifiée, même lorsque les fréquences d'horloges changeront.

Voici mon schéma de LED_driver, j'ai ajouté 3 nouvelles sorties pour piloter la deuxième led et j'ai remplacé la FIFO par un compteur 10 qui une fois fini redirigera le signal update et color_code vers LED_driver.



En vue des prochaines questions du projet et de l'utilisation d'une horloge différente pour chaque leds j'ai finalement opté pour une organisation à 2 instances LED0_driver et LED1_driver de l'entité LED_driver qui peuvent piloter chacune une led.



2. Rédigez le code VHDL correspondant à votre architecture.

Voir codes *LED_driver*, *pilote_led_driver* et *compteur_10*.

J'ai supprimé la machine à état et la FIFO de mes codes, je les ai remplacées par un système de changement de couleurs automatique (rouge, bleu, vert) en boucle. Le changement de couleur se fera à l'aide d'un compteur qui comptera le nombre de cycles allumés/éteints d'une led et changera de couleur tous les 10 cycles.

J'ai également instancié mes driver LED0 et LED1, qui pourront piloter les led0 et led1 indépendamment.

3. Rédigez le testbench et simulez votre design. Vérifiez que les modules réceptionnent correctement le signal update.

Voir code *tb_pilote_led_driver*.

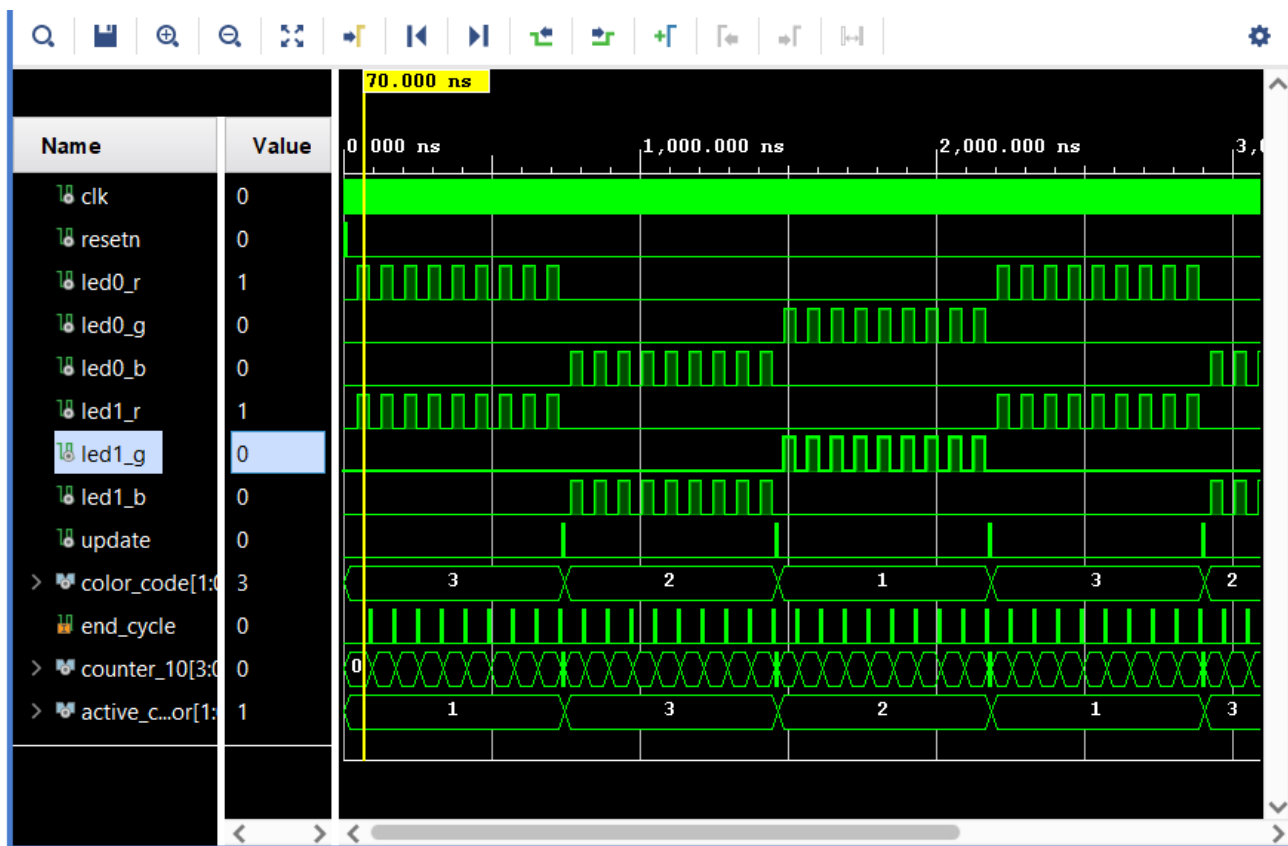
Pour ce testbench, j'ai testé si les leds s'allumaient bien au bon moment et dans la bonne séquence (rouge, bleu, vert). Avec la simulation on peut constater que c'est bien le cas.

Pour mes tests j'ai vérifié les emplacements suivants :

led0_r & led1_r allumés à 70ns

led0_b & led1_b allumés à 870ns

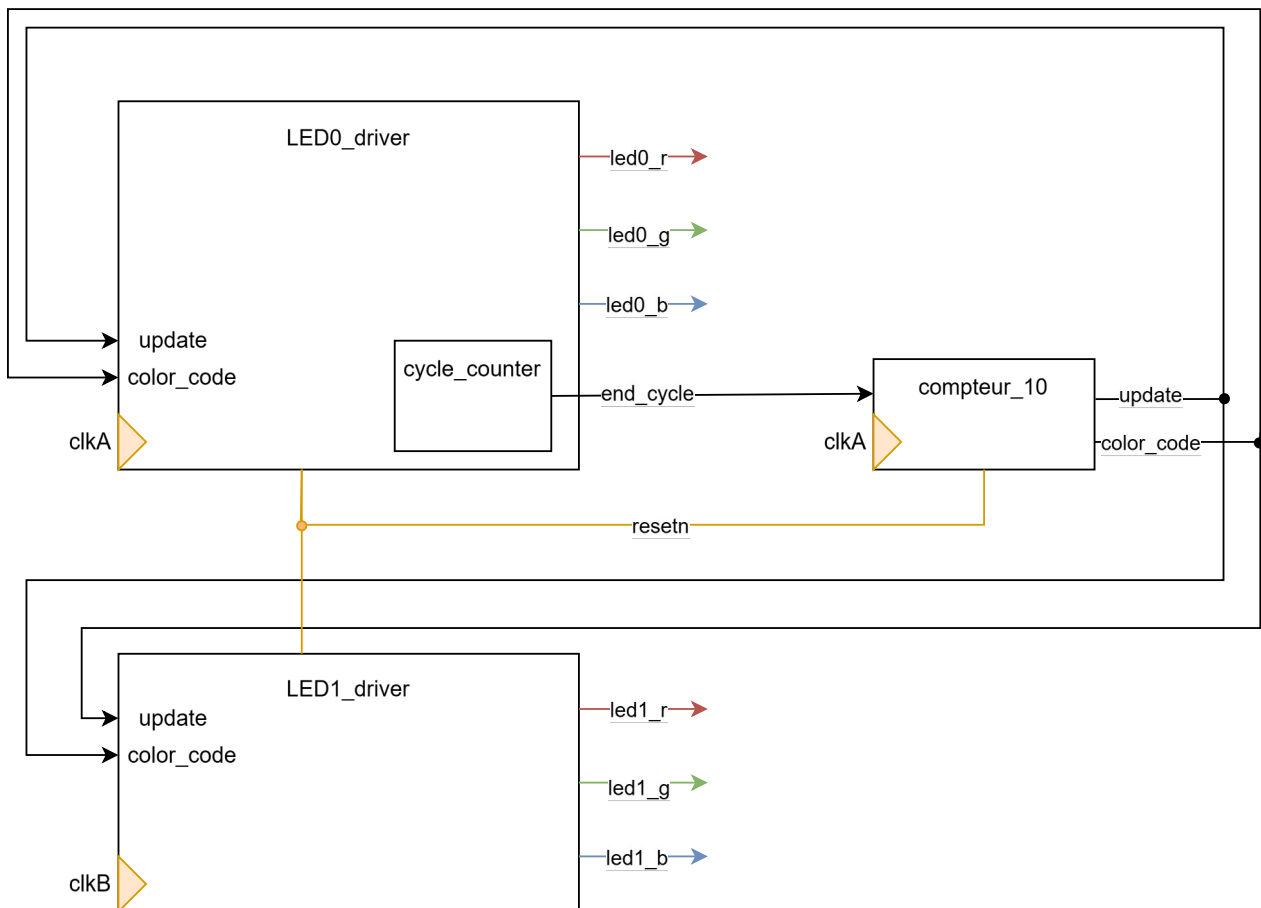
led0_g & led1_g allumés à 1740ns



4. Modifiez votre design pour gérer deux signaux d'horloge différents. La première horloge, clkA, est associé à la logique en dehors des modules LED_driver et au module LED_driver de la LED0. La deuxième horloge, clkB, est associé au module LED_driver de la LED1. Le changement de couleur des deux LEDs à lieu lorsque la LED0 à clignoté 10 fois.

Étant donné que j'avais déjà commencé un schéma avec un driver pour chaque led, je n'ai pas grand-chose à modifier par rapport à mon schéma RTL de la Q1.

Je n'ai donc changé que les horloges.



Et pour le code j'ai simplement ajouté une deuxième horloge à *pilote_led_driver*, et j'ai instancié mes modules avec les horloges correspondantes soit :

```
clkA → LED0_driver
clkA → compteur_10
clkB → LED1_driver
```

5. Modifiez votre testbench tel que l'horloge clkA ait une fréquence de 250Mz et clkB 50MHz.

Voir code *tb_top_led_driver*

J'ai modifié mon testbench pour qu'il possède une deuxième horloge, clkA et clkB, j'ai également modifié ma constante « period » pour qu'elle copie la valeur de l'horloge clkA, afin de ne pas avoir à changer mes tests pour le moment.

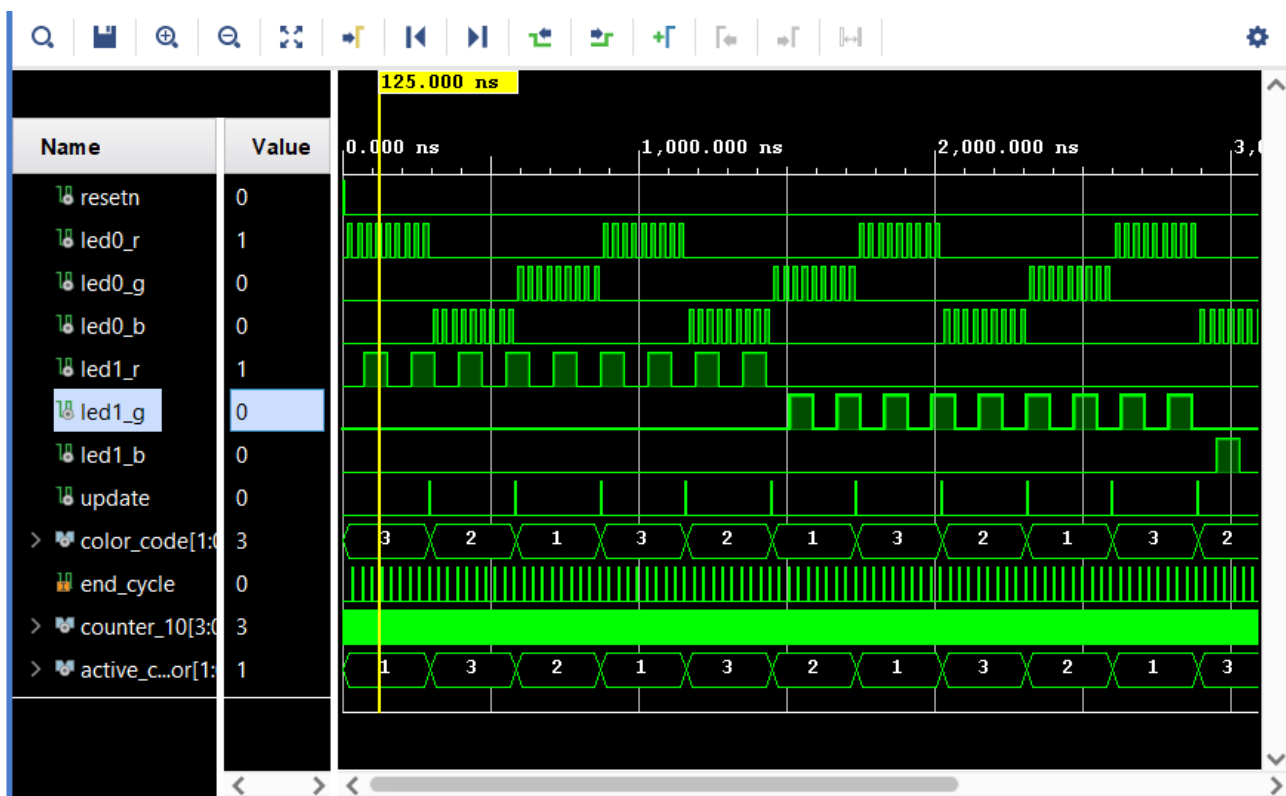
6. Lancer une simulation. Que se passe-t-il au niveau des signaux update des modules LED_driver ? Qu'elle incidence cela a-t-il sur les LEDs ?

En ajoutant une deuxième horloge au test bench, on obtient une simulation mais cela crée des problèmes avec le reste du code car après 10 périodes de clkA quand led0 change de couleur, la led1 ne change pas et attend les 10 clignotements de sa couleur.

Cela est problématique car la led1 est pilotée avec le même color_code que la led0. On a alors une des 2 led qui ne suit plus la séquence de base (rouge, bleu, vert).

Au début je pensais que la led0 avait bien une séquence de rouge, bleu, vert avec chacun 10 clignotements mais lorsque la led1 finissait ces 10 clignotements le color_code est en vert et la séquence de la led1 devenait donc rouge, vert, bleu.

En fait ce n'est pas que la led1 veut finir ces 10 clignotements avant de changer de couleur mais surtout que le LED1_driver regarde si update est à 1 que sur un front montant de l'horloge clkB, comme elle est beaucoup plus lente que clkA, elle manque la « rencontre » et reste de la même couleur, leur rencontre ne se produit que tous les 1.44 us.



7. Proposez une solution pour corriger le problème lié au changement de domaine d'horloge, vous pourrez vous aider du lien <https://nandland.com/lesson-14-crossing-clock-domains/>.

Il faudrait réussir à étirer le signal update afin que les 2 horloges puissent le lire à chaque fois. Dans notre cas il faudrait que le signal update dure au moins un peu plus qu'une période entière de l'horloge B pour être sûr qu'ils se coordonnent.

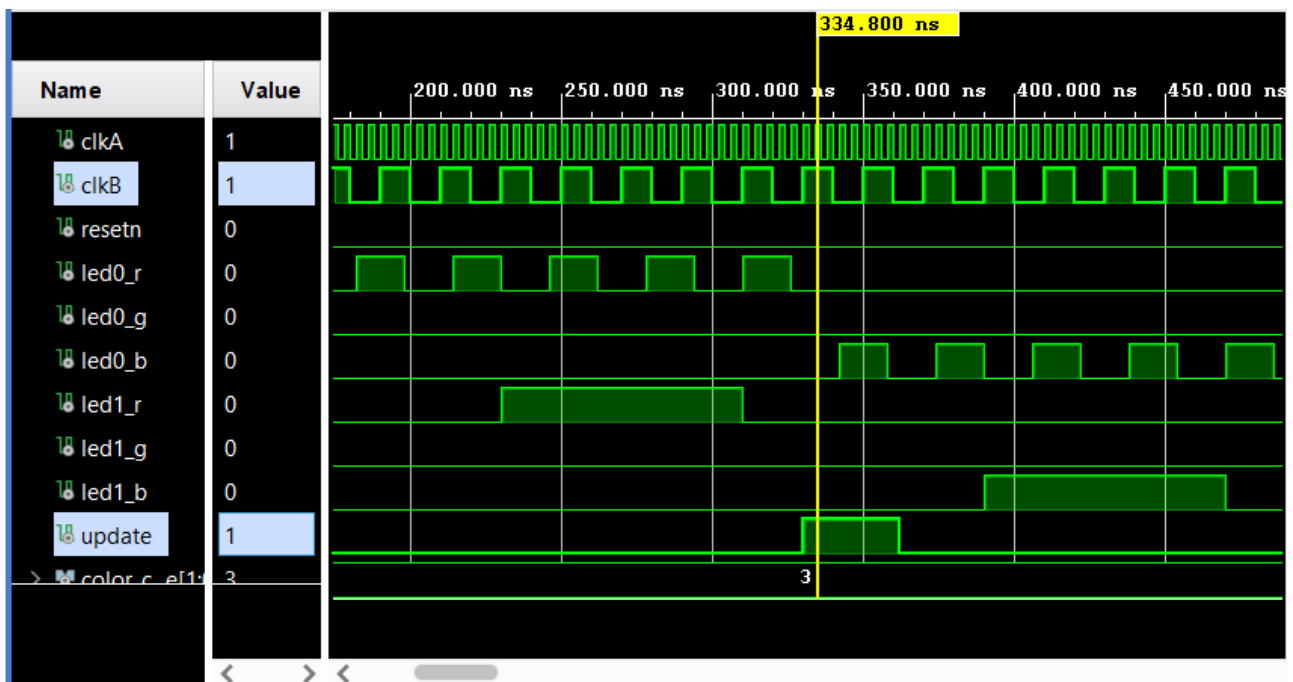
Il faudrait donc créer un étireur de signal, sous la forme d'un compteur, le signal débiterait lorsque update passe à 1 et s'étirerait sur une durée décidée à l'avance, dans notre cas : un peu plus d'une période de clkB.

8. Mettez en place votre solution et testez-la en simulation. Si votre résultat de simulation n'est toujours pas valide, proposez une autre solution.

Voir code *compteur_10* et *update_stretcher*

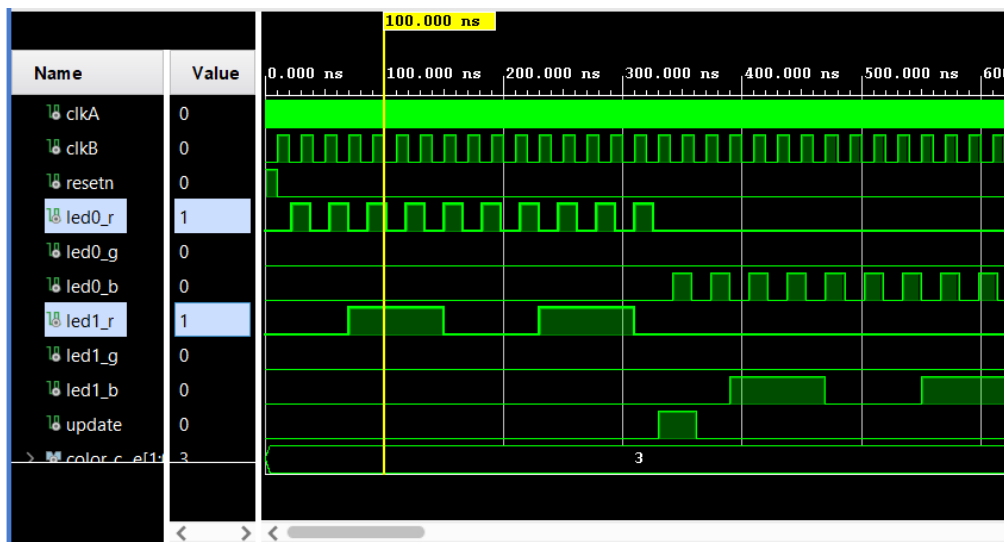
J'ai dû ajouter un signal *color_hold* afin de maintenir la prochaine couleur, dans certaines situations lors du début de la simulation il arrivait qu'une couleur soit « sautée » et cela pouvait passer du rouge au vert directement. Avec ce nouveau signal, le problème est totalement corrigé.

La solution marche correctement le signal update a bien été étiré et fait largement la taille d'une période de clkB, il n'aura donc aucune difficulté à détecter un front montant de clkB :

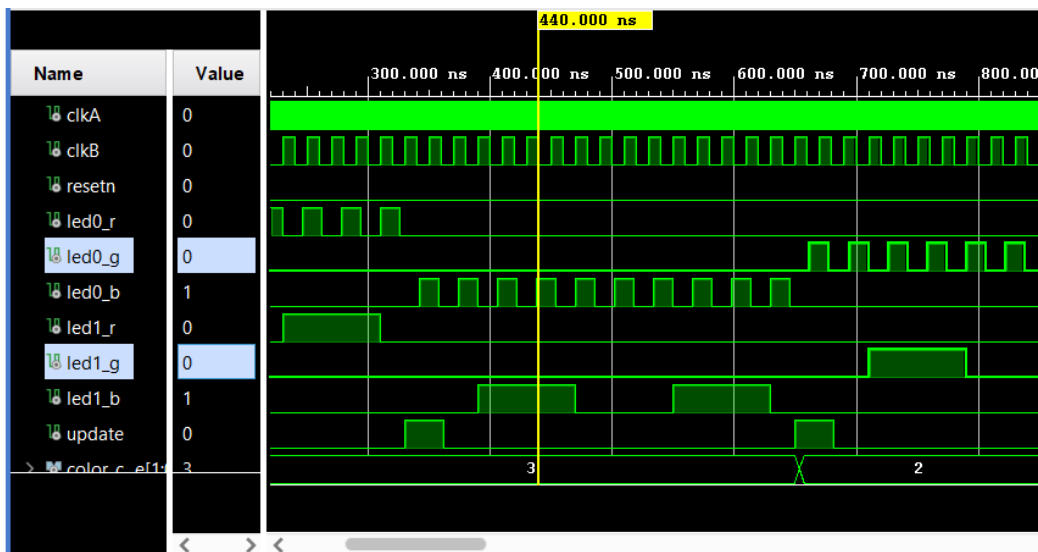


Pour les tests, je n'ai fait qu'un seul scénario comme le but est que les leds clignotent automatiquement :

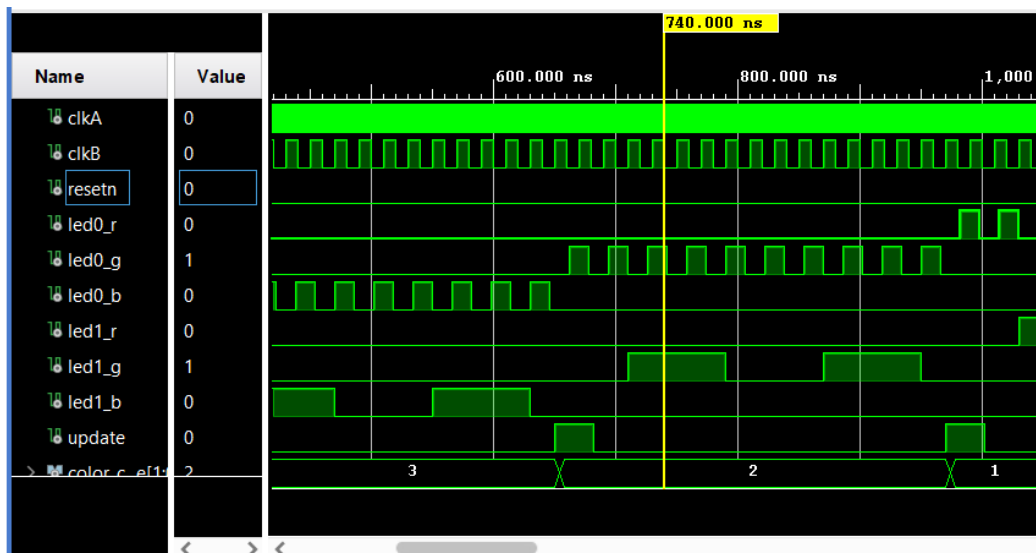
100 ns : led0_r & led1_r sont allumées



440 ns : led0_b & led1_b sont allumées



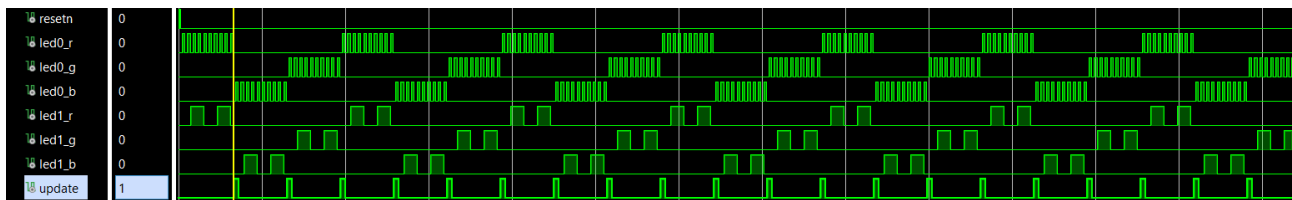
740 ns : led0_g & led1_g sont allumées



Et comme on peut le voir le cycle se répète indéfiniment et sans aucune erreurs, notre testbench est donc réussi !

```
Tcl Console x Messages Log
[Search] [Zoom In] [Zoom Out] [Run] [Copy] [Paste] [Delete]

# run 1000ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_pilote_led_driver_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:08 . Memory (MB): peak = 1597.816 ; gain = 0.000
run 2000
run 10000
< [Progress Bar]
Type a Tcl command here
```



9. Pour générer plusieurs horloges vous aurez besoin d'une PLL. Trouvez quel est le nom de l'IP PLL de l'IP Catalog de Vivado puis ajoutez là à votre architecture.

La fréquence de l'horloge en entrée de la PLL est de 100MHz. Les deux horloges en sortie doivent être à 50MHz et 250MHz.

J'ai utilisé une PLL pour générer mes deux autres horloges, mais qu'est-ce qu'une PLL ?

PLL (Phase-Locked Loop / Boucle à verrouillage de phase) :

Une PLL est un circuit qui prend une horloge de référence en entrée et avec cette dernière elle peut créer plusieurs autres horloges, pour ce projet 2 horloges suffisent.

Notre PLL est composé de 5 ports :

- clk_in1** : l'horloge de référence, ici réglée sur 100MHz.
- reset** : un signal pour forcer la PLL à se recalibrer, pour mon code je l'ai laissé à 0 afin que la PLL se calibre toute seule et n'ait pas à subir de redémarrage forcé
- clk_out1** : un signal d'horloge créé par la PLL, ici réglé sur 50MHz et utilisé pour clkB.
- clk_out2** : un deuxième signal d'horloge créé par la PLL, ici réglé sur 250MHz et utilisé pour clkA.
- locked** : un signal de « vérification » qui s'active qu'une fois que les signaux d'horloge sont stables, plus il y a d'horloges et plus elles sont différentes, plus le signal locked peut mettre du temps à s'activer

10. Effectuez la synthèse et placer des sondes (ILA) sur les signaux pertinents pour vérifier que le changement de domaine d'horloge s'est correctement passé.

Pour les sondes ILA j'ai choisi de passer par le Set Up Debug, et pour ce faire il faut marquer les signaux à observer.

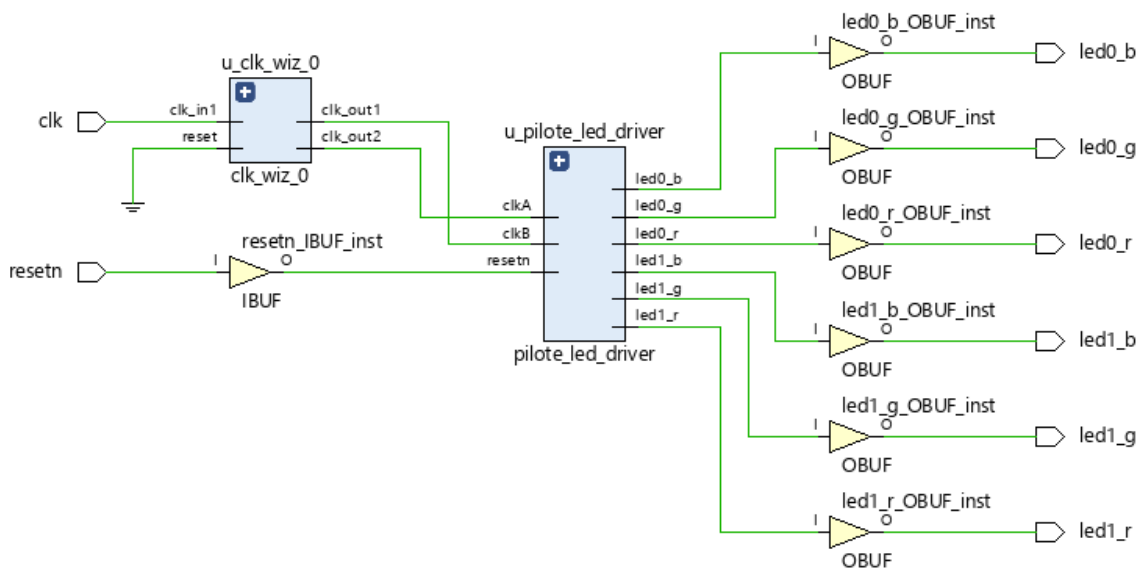
J'ai donc utilisé l'attribut « mark_debug » dans mon code et j'ai marqué des signaux dans les 2 domaines d'horloges afin de vérifier que le changement de domaine d'horloge se passe correctement.

clkA 250 MHz : update, color_code, end_cycle, led0_r, led0_g et led0_b
clkB 50 MHz : led1_r, led1_g et led1_b

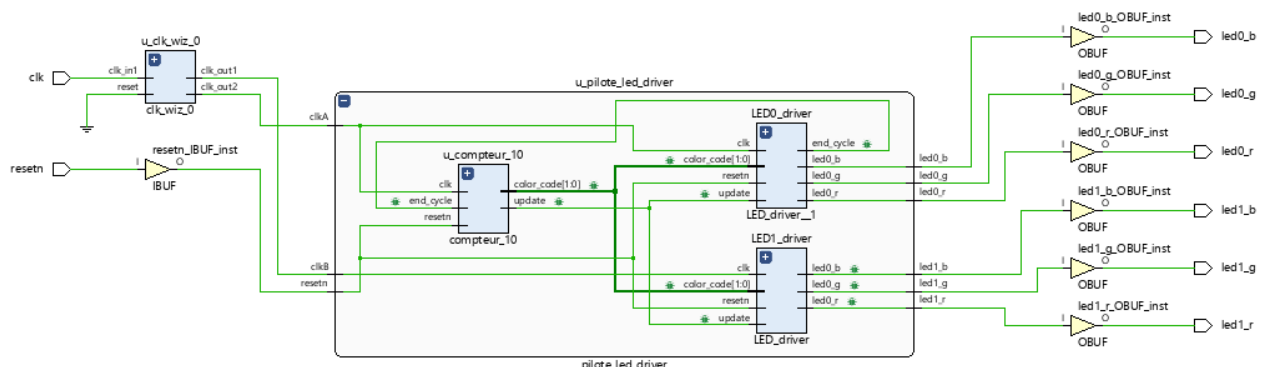
11. Etudiez le rapport de synthèse.

Après avoir généré la synthèse, je peux observer le schéma final et il est cohérent avec toutes les modifications que j'ai effectuées :

pilote_led_driver qui est piloté par la PLL, l'horloge de base entre uniquement dans la PLL et clkA et clkB sont en entrée de *pilote_led_driver*.
Les 6 sorties pour chaque couleur des 2 leds

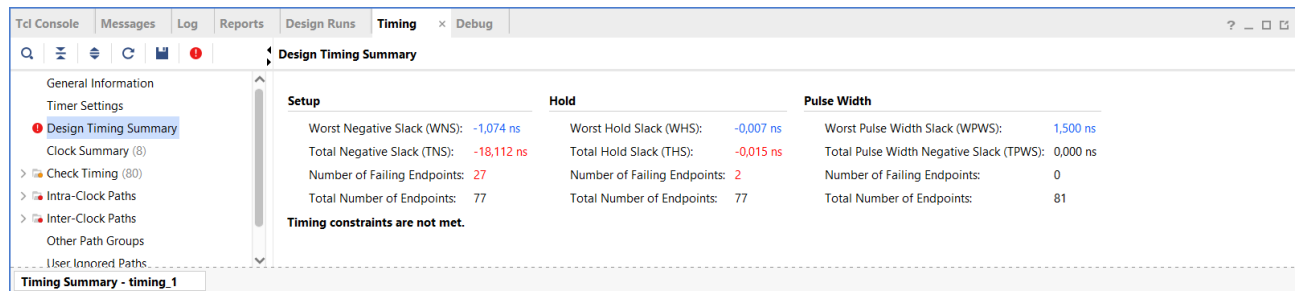


En dépliant le composant de *pilote_led_driver* on retrouve bien les schémas RTL que j'ai pu fournir plus tôt, les 2 leds sont pilotées par des instances différentes, de plus seul le `end_cycle` de `LED0_driver` est connecté au `compteur_10` qui renvoie bien le code couleur dans les deux instances de `LED_driver`.



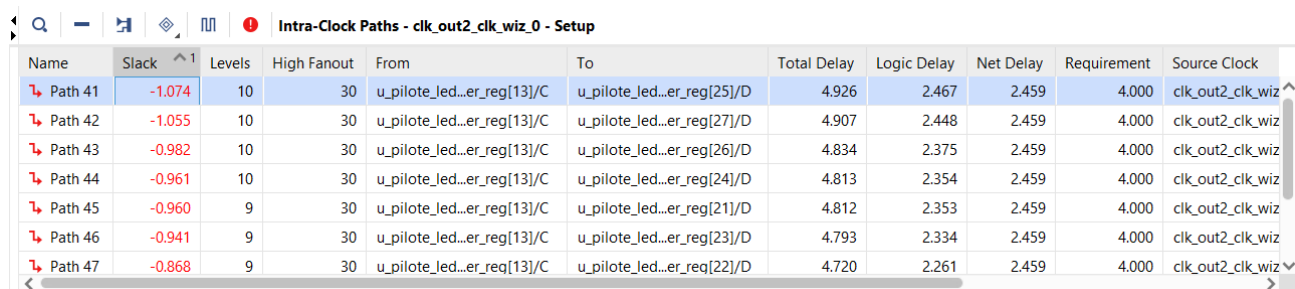
En étudiant les rapports de timing, je remarque tout de suite un slack(marge) négative. Cela peut arriver lorsqu'on utilise plusieurs domaines d'horloges, Vivado va comparer des chemins qui n'ont pas lieu d'être.

Mais ici le plus problématique c'est les 2 erreurs Hold.



Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -1,074 ns	Worst Hold Slack (WHS): -0,007 ns	Worst Pulse Width Slack (WPWS): 1,500 ns
Total Negative Slack (TNS): -18,112 ns	Total Hold Slack (THS): -0,015 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 27	Number of Failing Endpoints: 2	Number of Failing Endpoints: 0
Total Number of Endpoints: 77	Total Number of Endpoints: 77	Total Number of Endpoints: 81

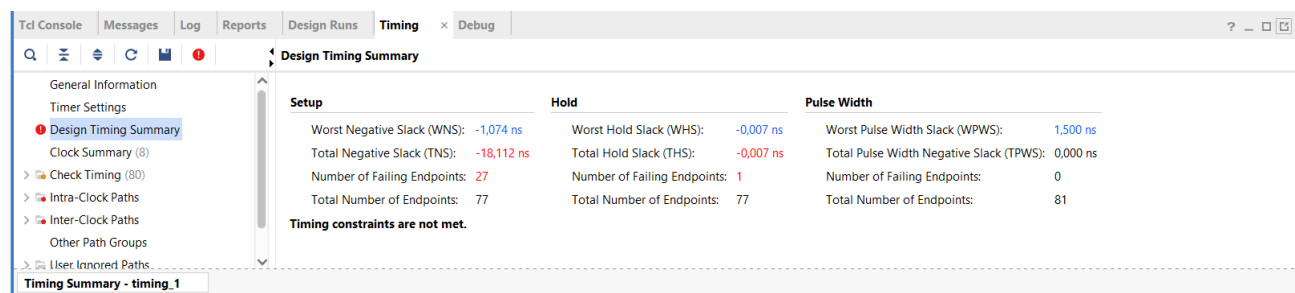
Timing constraints are not met.



Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
Path 41	-1.074	10	30	u_pilote_led...er_reg[13]/C	u_pilote_led...er_reg[25]/D	4.926	2.467	2.459	4.000	clk_out2_clk_wiz
Path 42	-1.055	10	30	u_pilote_led...er_reg[13]/C	u_pilote_led...er_reg[27]/D	4.907	2.448	2.459	4.000	clk_out2_clk_wiz
Path 43	-0.982	10	30	u_pilote_led...er_reg[13]/C	u_pilote_led...er_reg[26]/D	4.834	2.375	2.459	4.000	clk_out2_clk_wiz
Path 44	-0.961	10	30	u_pilote_led...er_reg[13]/C	u_pilote_led...er_reg[24]/D	4.813	2.354	2.459	4.000	clk_out2_clk_wiz
Path 45	-0.960	9	30	u_pilote_led...er_reg[13]/C	u_pilote_led...er_reg[21]/D	4.812	2.353	2.459	4.000	clk_out2_clk_wiz
Path 46	-0.941	9	30	u_pilote_led...er_reg[13]/C	u_pilote_led...er_reg[23]/D	4.793	2.334	2.459	4.000	clk_out2_clk_wiz
Path 47	-0.868	9	30	u_pilote_led...er_reg[13]/C	u_pilote_led...er_reg[22]/D	4.720	2.261	2.459	4.000	clk_out2_clk_wiz

Afin de corriger les erreurs de Hold j'ai ajouté une ligne dans le fichier contrainte pour que Vivado comprenne que les deux horloges tournent de manière asynchrone.

```
set_clock_groups -asynchronous -group [get_clocks clk_out1_clk_wiz_0] -group [get_clocks clk_out2_clk_wiz_0];
```



Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -1,074 ns	Worst Hold Slack (WHS): -0,007 ns	Worst Pulse Width Slack (WPWS): 1,500 ns
Total Negative Slack (TNS): -18,112 ns	Total Hold Slack (THS): -0,007 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 27	Number of Failing Endpoints: 1	Number of Failing Endpoints: 0
Total Number of Endpoints: 77	Total Number of Endpoints: 77	Total Number of Endpoints: 81

Timing constraints are not met.

Comme le problème était encore présent, on voit sur la capture d'écran précédente qu'il y a toujours une erreur dans le Hold.

J'ai ensuite remarqué que Vivado avait créé plusieurs « objets » de chaque horloges :
clk_out_clk_wiz_0 et clk_out_clk_wiz_0_1

Intra-Clock Paths - clk_out2_clk_wiz_0

Clock: clk_out2_clk_wiz_0

Statistics

Type	Worst Slack	Total Violation	Failing Endpoints	Total Endpoints
Setup	-1,074 ns	-18,112 ns	27	46
Hold	0,142 ns	0,000 ns	0	46
Pulse Width	1,500 ns	0,000 ns	0	44

Timing Summary - timing_1

J'ai donc ajouté un * après le nom des horloges dans mon fichier contrainte pour qu'il prenne bien tous les objets avec un nom similaire :

```
set_clock_groups -asynchronous -group [get_clocks clk_out1_clk_wiz_0*] -group [get_clocks clk_out2_clk_wiz_0*];
```

Suite à cette dernière modification, je n'ai plus d'erreur dans mon Hold.

Et j'ai corrigé des petites erreurs que je n'avais pas vu, cela à grandement fait baisser le nombre d'erreurs et la marge négative :

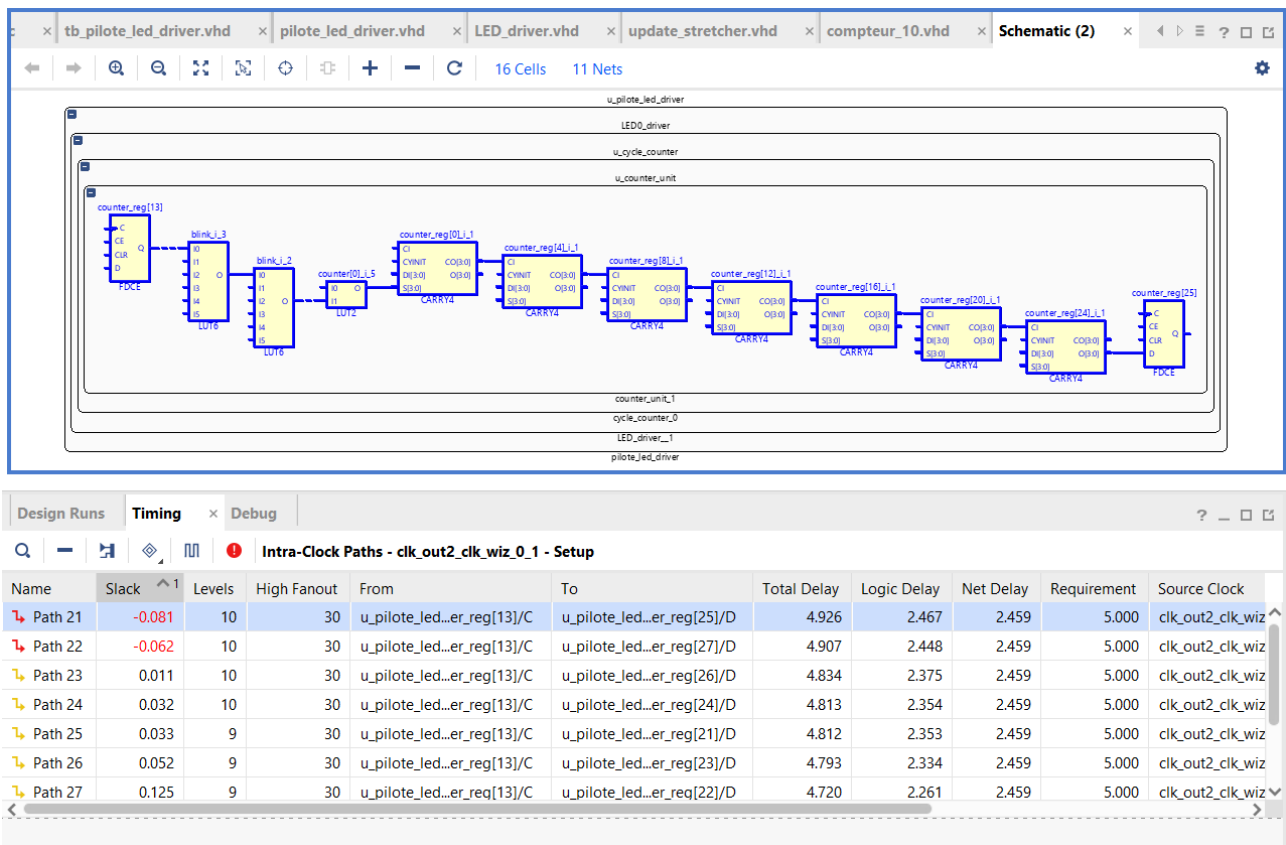
Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -0,081 ns	Worst Hold Slack (WHS): 0,142 ns	Worst Pulse Width Slack (WPWS): 2,000 ns
Total Negative Slack (TNS): -0,142 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 2	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 78	Total Number of Endpoints: 78	Total Number of Endpoints: 82

Timing constraints are not met.

Timing Summary - timing_1

Pour la synthèse j'obtiens un chemin critique à -0,081 ns dans le Setup :



C'est un chemin de *counter_unit* entre le bit 13 et le bit 25 du compteur, avec la synthèse seule (pré-routage/implémentation) la logique de comparaison du compteur est tout juste trop lente pour tenir les 250MHz.

Pour le rapport d'utilisation on ne remarque aucun problème.

8 Bonded IOB : 2 entrées clk et resetn, 6 sorties pour les 3 couleurs des deux leds.

Toujours 28 registre dans le *counter_unit*.

10 registres dans le compteur_10 : 4 bits pour compter jusqu'à 10, 2 bits pour le color_code et 4 bits du *update_stretcher*.

4 registres dans le *update_stretcher* : 3 bits pour compter jusqu'à 6 et 1 bit pour retenir le signal (hold).

Cette fois on a un BUFGCTRL à 3 et c'est normal puisqu'on utilise 3 horloges différentes, enfin c'est ce que je croyais mais en fait il y a les 2 horloges clkA et clkB et un signal de rétroaction donc 3.

Q

≡

⚙

%

Hierarchy

Name	1	Slice LUTs (17600)	Slice Registers (35200)	Bonded IOB (100)	BUFGCTRL (32)	PLLE2_ADV (2)
▼ N top_clock		89	73	8	3	1
> I u_clk_wiz_0 (clk_wiz_0)		0	0	0	3	1
▼ I u_pilote_led_driver (pilote_led_driver)		89	73	0	0	0
> I LED0_driver (LED_driver_1)		40	32	0	0	0
▼ I LED1_driver (LED_driver)		40	31	0	0	0
▼ I u_cycle_counter (cycle_counter)		38	29	0	0	0
I u_counter_unit (counter_unit)		36	28	0	0	0
▼ I u_compteur_10 (compteur_10)		9	10	0	0	0
I u_update_stretcher (update_stretch)		3	4	0	0	0

12. Effectuez le placement routage et étudiez les rapports générés

Après avoir effectué le routage j'ai observé les rapports de timing et d'utilisation.

Le rapport de timing à quelques peu évolué, le slack est maintenant positif ! On a le message « All user specified constraints are met »

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.497 ns	Worst Hold Slack (WHS): 0.031 ns	Worst Pulse Width Slack (WPWS): 1.520 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 5624	Total Number of Endpoints: 5608	Total Number of Endpoints: 3286

All user specified timing constraints are met.

Le chemin critique est maintenant à 0,407 ns, c'est toujours un chemin de *counter_unit* mais cette fois il va du bit 22 au bit 25 et il n'est plus négatif, la logique de comparaison du compteur et donc parfaitement dans les temps :

Design Runs | Power | DRC | Methodology | Timing

Intra-Clock Paths - clk_out2_clk_wiz_0_1 - Setup

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
Path 21	0.497	10	30	u_pilote_led...er_reg[22]/C	u_pilote_led...er_reg[25]/D	4.468	2.133	2.335	5.000	clk_out2_clk_wiz
Path 22	0.507	3	8	<hidden>	<hidden>	4.121	1.473	2.648	5.000	clk_out2_clk_wiz
Path 23	0.518	10	30	u_pilote_led...er_reg[22]/C	u_pilote_led...er_reg[27]/D	4.447	2.112	2.335	5.000	clk_out2_clk_wiz
Path 24	0.592	10	30	u_pilote_led...er_reg[22]/C	u_pilote_led...er_reg[26]/D	4.373	2.038	2.335	5.000	clk_out2_clk_wiz
Path 25	0.608	10	30	u_pilote_led...er_reg[22]/C	u_pilote_led...er_reg[24]/D	4.357	2.022	2.335	5.000	clk_out2_clk_wiz
Path 26	0.616	9	30	u_pilote_led...er_reg[12]/C	u_pilote_led...er_reg[21]/D	4.350	2.019	2.331	5.000	clk_out2_clk_wiz
Path 27	0.637	9	30	u_pilote_led...er_reg[12]/C	u_pilote_led...er_reg[23]/D	4.329	1.998	2.331	5.000	clk_out2_clk_wiz

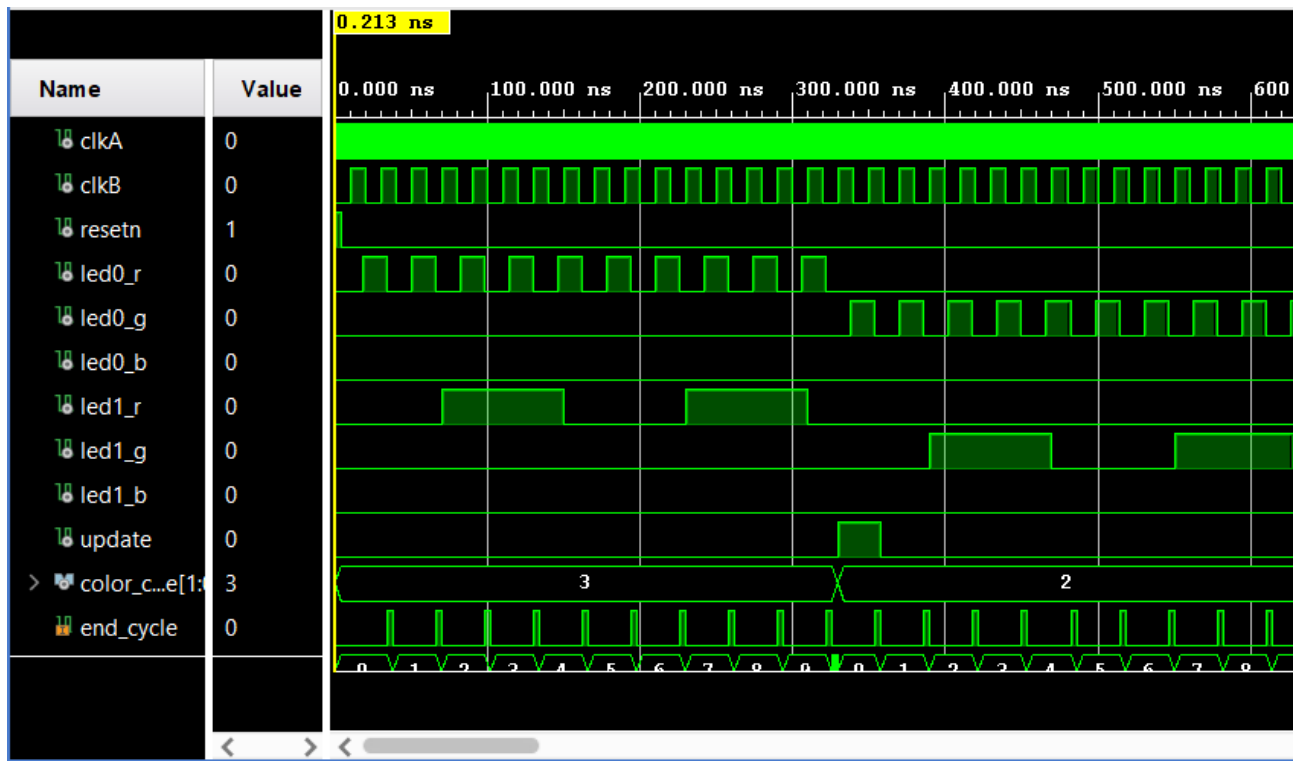
Pour le rapport d'utilisation il n'a quasiment pas changé.

Hierarchy								
Name	1	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Bonded IOB (100)	BUFGCTRL (32)	PLLE2_ADV (2)
▼ N top_clock		90	73	35	90	8	3	1
▼ I u_clk_wiz_0 (clk_wiz_0)		0	0	0	0	0	3	1
I inst (clk_wiz_0_clk_wiz_0_clk_wiz)		0	0	0	0	0	3	1
▼ I u_pilote_led_driver (pilote_led_driver)		90	73	35	90	0	0	0
> I LED0_driver (LED_driver__1)		42	32	17	42	0	0	0
▼ I LED1_driver (LED_driver)		40	31	15	40	0	0	0
> I u_cycle_counter (cycle_counter)		38	29	15	38	0	0	0
▼ I u_compteur_10 (compteur_10)		8	10	5	8	0	0	0
I u_update_stretcher (update_stretc		3	4	2	3	0	0	0

Il n'y a pas de très gros changement entre les rapports avant et après implémentation, le plus gros changement est l'amélioration de la marge et du nombre d'erreurs ce qui est une bonne chose.

13. Générez le bitstream, programmez la carte et vérifiez les signaux du chipscope (ILA).

Après avoir généré le bitstream et observé le programme sur la carte j'ai remarqué que ma led0 pilotée par l'horloge la plus rapide ne clignotait que 9 fois et ce depuis le début, cela m'avait échappé lors des simulations. J'ai compris que lorsque mon compteur arrivait à 10 il provoquait le changement de couleur mais ne laissait pas la led effectuer son dernier clignotement, je l'ai donc passé en compteur 11 et on observe bien les 10 clignotements sur la simulation comme sur la carte en physique.



Les leds clignotent bien selon leurs horloges respectives et change de couleurs en même temps une fois que la led pilotée par l'horloge la plus rapide a effectué 10 clignotements en passant du rouge au bleu, du bleu au vert et du vert au rouge en boucle (voir vidéo de démonstration_1).

De plus tous les signaux du chipscope ILA change bel et bien en temps réel, le end_cycle se déclenche à chaque clignotement de la led0 et j'ai fait un exemple : le update se déclenche au moment précis où la led0 changent de couleurs (voir vidéo de démonstration_ila_update).

Et le même phénomène se produit pour les signaux led1_r/g/b, au moment exact où la led1 s'allume le signal chipscope ILA se déclenche, j'ai fait un exemple avec la led1_b, en réglant la consigne sur 1 on voit le signal se déclencher au premier clignotement de la led1_b (voir vidéo de démonstration_ila_led1).